Implementation Plan & Engineering Roadmap

AI-Driven Social Media Audience Intelligence Framework

Academic Research Prototype | 7th Semester CSE Minor Project | Team of 4 (P1–P4) | Sep – Dec 2026

Reference Hardware: NVIDIA GTX 1650 Ti (4GB VRAM) | 16GB RAM | Offline-First (v4.8)

> **Academic Research Testbed Disclaimer:**
> This system is an undergraduate academic proof-of-concept (PoC) and research testbed developed for a 7th-semester CSE minor project. The National Technical Research Organisation (NTRO) is the origin of the public problem statement **only**. This software is **not** an operational intelligence system, is **not** deployed by NTRO or any defense agency, and does **not** claim operational intelligence capability.

---

1. Executive Summary & Project Mandate

- **Project Title:** AI-Driven Social Media Audience Intelligence Framework (Academic Prototype)
- **Problem Origin:** NTRO (National Technical Research Organisation), India — Public Social Media Audience Intelligence Problem Statement (Academic Inspiration Only)
- **Core Mission:** Deliver an academic research testbed capable of monitoring a curated set of 30 public Telegram channels, processing multi-dialectal and code-mixed (English + Hinglish) discourse with off-the-shelf pre-trained transformers, extracting macro-demographic heuristic trends ($k \ge 50$), screening for coordinated inauthentic behavior (CIB) via a 3-signal anomaly heuristic, and answering analyst inquiries via a grounded, short-circuiting Retrieval-Augmented Generation (RAG) interface over agent summaries.
- **The Cardinal Product Rule (Offline-First Invariant):**
  "Build the offline demo as the primary product; treat live Telegram collection and cloud LLMs as optional depth."
  Campus Wi-Fi drops, Telegram FloodWait rate limits, and missing API keys are fatal demo failure modes. The platform's primary deliverable is a deterministic, zero-internet offline seed data replay (`backfill.py --offline` / SQL seed) and air-gapped local LLM inference via Ollama (`llama3.2:3b`). Live Pyrofork MTProto ingestion is a secondary showcase if connectivity permits.
- **Reference Development & Demo Machine:**
- **GPU:** NVIDIA GeForce GTX 1650 Ti Mobile (4 GB GDDR6 VRAM)
- **System Memory:** 16 GB DDR4 RAM
- **Storage:** 1 TB NVMe SSD + 1 TB HDD
- **OS:** Windows 11 / WSL2 / Docker Desktop
- **Teammate Fallback Minimum Spec:** 8–16 GB RAM, Quad-Core CPU, CPU-only fallback using quantized models / seed replay.

- **The Demo Timing Invariant:** The "Run Analysis Now" live demo trigger is guaranteed to complete in **15–25 seconds on the GTX 1650 Ti** for a strictly capped batch of **≤ 50 seed posts**. Continuous asynchronous processing runs on a 15-minute Celery Beat schedule.

---

2. Team Responsibility Matrix (P1–P4)

The project decomposes cleanly across four teammates, ensuring clear ownership, zero blocking bottlenecks, and total defensibility during institutional reviews and external viva examination:

Member	Primary Modules & Focus	Key Deliverables & Code Packages	Technologies & Libraries
P1	**Data Ingestion, Storage & Infrastructure**	`ingestion_a/`, `storage/`, `docker-compose.yml`, offline seed replay scripts	Python 3.11, Pyrofork (MTProto), PostgreSQL 16, pgvector, Redis, Celery, Docker
P2	**NLP, Sentiment, Stance & Topic Discovery**	`sentiment_b/`, `trend_d/`, sequential model runner, velocity modeling	Hugging Face Transformers (`twitter-xlm-roberta-base-sentiment`, `go_emotions`, `distilbart-mnli-12-3`), BERTopic, UMAP, HDBSCAN
P3	**Network Analytics, CIB Heuristic & Orchestration**	`network_e/`, `orchestrator/`, interaction graph construction, dual runners	NetworkX, `python-louvain` (`community`), FastAPI background tasks, Celery Beat
P4	**Demographics, RAG Engine & Analyst UI**	`demographic_c/`, `rag/`, `frontend/`, k-anonymity guards, Bento Grid	React 18, Vite, Recharts, `react-force-graph`, Sentence-Transformers, Ollama / Groq

---

3. Five-Phase Implementation Timeline (Weeks 1–15)

```
Week:    1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
P1:     [--- Phase 1 ---]                               [-- Phase 5 --]
P2:                     [--- Phase 2 ---]               [-- Phase 5 --]
P3:                     [--- Phase 2 ---][-- Phase 3 --][-- Phase 5 --]
P4:                                      [-- Phase 3 --][-- Phase 4 --]
Review:         ▲ Checkpoint 1          ▲ Checkpoint 2  ▲ Checkpoint 3  ▲ Viva
```

Phase 1: Ingestion Engine & Storage Foundation (Weeks 1–4)

Lead Owner: P1 (all teammates review & test locally)

- **Objectives:** 1. Finalize target channel list in `config/channels.yaml` (30 curated public broadcast channels covering Indian space, defense, strategic affairs, and tech policy). 2. Implement `BaseIngestionAdapter` interface and `TelegramIngestionAdapter` using **Pyrofork MTProto**. 3. Enforce staggered MTProto rate limiting: 1 channel polled every 30 seconds with 2–5s randomized jitter across a 15-minute cycle ($30 \times 30\text{s} = 900\text{s}$), maintaining a safe global rate of ~ 2 requests/min to strictly eliminate Telegram API FloodWait. 4. Pin Celery worker concurrency for ingestion to `--concurrency=1` to prevent SQLite `.session` database lock contention. 5. Setup Docker Compose containing PostgreSQL 16 with `pgvector` extension and

Redis. 6. Execute Alembic migration `0001_initial.py` establishing relational schemas (`raw_posts` , `raw_authors` , `ingestion_runs`) with vector extension initialization. 7. **Primary Product Deliverable:** Implement offline seed replay script (`ingestion_a/backfill.py --offline`) loading sanitized historical posts from `scripts/seed_demo_data.sql` with normalized UTC timestamps. 8. Implement Telegram Desktop chat export parser (`ingestion_a/import_exported.py --import-json <path>`) for bulk historical backfill.

- **Milestone / Checkpoint 1 Deliverable:** Docker Compose starts cleanly; offline seed replay populates the database in under 5 seconds without internet; live MTProto ingestion successfully captures posts when network is present.

Phase 2: Core NLP, Stance & Topic Modeling with 4GB VRAM Discipline (Weeks 5–8)

Lead Owners: P2 (Component B & D) and P3 (Orchestrator Row-Level Skeleton)

- **Objectives:** 1. Implement **Component B (Sentiment, Emotion & Stance)** with off-the-shelf pre-trained transformers:
  - Multilingual 3-class sentiment via `cardiffnlp/twitter-xlm-roberta-base-sentiment` across all posts (English and code-mixed Hinglish).
  - Fine-grained emotion via `SamLowe/roberta-base-go_emotions` , mapped deterministically into Ekman 6 core dimensions (`joy` , `anger` , `sadness` , `disgust` , `surprise` , `fear`) + `neutral` .
  - Zero-Shot stance detection via `valhalla/distilbart-mnli-12-3` (~260 MB, runtime locked) against configured policy topics with a strict ≥ 0.60 threshold (defaulting to `'neutral'` or `'unclear'`).
  - Advisory sarcasm heuristic (valence-emoji incongruity) setting `sarcasm_flag = TRUE` without inverting polarity.
  - Synthesis of a 2–4 sentence factual analytical summary per post. 2. **Enforce 4GB VRAM Sequential Execution Invariant:**
  - Total model footprint (~4.3 GB) exceeds GTX 1650 Ti 4GB VRAM if co-resident.
  - The worker runs models sequentially: XLM-RoBERTa $\rightarrow$ GoEmotions $\rightarrow$ DistilBART $\rightarrow$ MiniLM embeddings, calling `torch.cuda.empty_cache()` between steps to prevent CUDA OOM crashes. 3. Implement **Component D (Trend & Velocity Engine)**:
  - Neural topic modeling via BERTopic using multilingual embeddings (`paraphrase-multilingual-MiniLM-L12-v2`).
  - Clustering with HDBSCAN (`min_topic_size = 3`) and outlier reduction via c-TF-IDF. Cap input batch at 500 documents per run.
  - Calculate hourly topic velocity ($v = \frac{\Delta \text{posts}}{\Delta t}$) and acceleration ($a = \frac{\Delta v}{\Delta t}$) strictly anchored to platform `posted_at` . 4. Wire the **Orchestrator Row-Level Runner**:
  - Polls `raw_posts WHERE processing_status = 'pending'` , fires Component B, and commits results to `sentiment_results_b` .
  - Provides instant demo trigger processing ≤ 50 posts in 15–25 seconds on GTX 1650 Ti.
- **Milestone / Checkpoint 2 Deliverable:** Seed posts processed through Component B and D on local GPU without OOM; database contains sentiment, stance, and trend velocity metrics; Swagger UI demonstrates live endpoint execution.

Phase 3: Demographics, Network Interaction & CIB Anomaly Detection (Weeks 9–11)

Lead Owners: P3 (Component E) and P4 (Component C)

- **Objectives:** 1. Implement **Component C (Demographic Profiling Heuristic)**:
  - Broadcast-level metadata heuristic extraction (language distribution, diurnal posting rhythms).
  - Audience-level bio-text heuristic keyword mining on active commenters in linked public discussion groups.
  - **Enforce DPDP Act 2023 k-Anonymity:** Suppress any demographic cluster smaller than 50 accounts (`below_threshold: true`). Coarse proxy, not census profiling.
  - Store output in `demographic_aggregates_c` . 2. Implement **Component E (Network Analytics & CIB Heuristics)**:
  - Construct directed interaction graph (forwards, mentions, replies) using NetworkX.
  - Apply Louvain community modularity (`community-louvain` undirected projection) for behavioral partition.
  - Compute 3-Signal CIB anomaly screening score: $$\text{Anomaly Score} = \frac{s_1 + s_2 + s_3}{3}$$ where $s_1 = \text{Posting Synchrony}$, $s_2 = \text{Account Age/Volume Incongruity}$, $s_3 = \text{Duplicate Content Similarity}$.
  - Flag coordinated behavior signal when $\text{Anomaly Score} \ge 0.70$ (screening signal only, never a legal verdict).
  - Cap exported network graph payload to top 500 nodes by betweenness centrality to protect browser Canvas performance. 3. Wire the **Window-Level Orchestrator Runner**:
  - Executes Components C, D, and E across rolling 48-hour temporal windows.
- **Milestone / Checkpoint 3 Deliverable:** All 5 analytical agents operational; demographic clusters respect k-anonymity; network graph with community partitioning and CIB highlights viewable via API.

Phase 4: Grounded Vector RAG & Frontend Dashboard (Weeks 11–13)

Lead Owners: P4 (RAG Pipeline & Frontend) and P1 (System Integration)

- **Objectives:** 1. Implement **Vector Storage & Retrieval (RAG Layer)**:
  - Vectorize agent intelligence summaries using `sentence-transformers` (`paraphrase-multilingual-MiniLM-L12-v2` , 384-dimensional dense vectors).
  - Store in `rag_documents` indexed via PostgreSQL `pgvector` HNSW cosine index.
  - Enforce strict cosine distance gating: if nearest document distance > 0.65, short-circuit with safe abort to guarantee zero hallucination.
  - Implement dual LLM routing: **Ollama (`llama3.2:3b`) as offline viva primary**, with optional Groq (`llama-3.3-70b-versatile`) when internet and API key are available. (No unverified "98% accuracy" marketing claims).
  - Log all analyst queries, timestamps, and retrieved document references to `rag_query_log` for audit compliance. 2. Build the **React 18 + Vite Operations Dashboard**:
  - Adopt Dark Utilitarian Minimalism (`#090d16` canvas, `#0f172a` cards, 1px `#1e293b` borders, JetBrains Mono telemetry).
  - Implement Bento Grid modular layouts across 7 core views: 1. `Overview.jsx` : Executive telemetry, system health, and "Run Analysis Now" live demo trigger button (≤ 50 posts in 15–25s). 2. `SentimentTimeline.jsx` : Multi-series Recharts displaying emotion distributions and policy stance gauges. 3. `TrendExplorer.jsx` : Velocity and acceleration ranked table with sparklines. 4. `Demographics.jsx` : Aggregate

audience persona cards with grayed-out k-anonymity badges for sub-threshold clusters. 5. `NetworkGraph.jsx`: Interactive 60 FPS WebGL graph (`react-force-graph-2d`) with CIB anomaly highlighting. 6. `DataSources.jsx`: Monitored Telegram channels, Pyrofork sync health, and jitter status. 7. `QueryPanel.jsx`: Analyst intelligence terminal with citation inspection drawer and confidence meters.

- **Milestone / Checkpoint 4 Deliverable:** Fully responsive 7-view web application running locally on port 5173, communicating seamlessly with FastAPI backend on port 8000.

Phase 5: Empirical Benchmarking, Hardware Validation & Viva Defense (Weeks 13–15)

All Team Members (P1–P4)

- **Objectives:** 1. Execute offline benchmark evaluations:
  - Component B: Sentiment accuracy against TweetEval benchmark split; compare DistilBART vs BART-large inference latency on GTX 1650 Ti.
  - Component E: CIB heuristic evaluation against the Kaggle IOA ground-truth dataset.
  - RAG: Grounding and retrieval evaluation against 10 gold-standard intelligence questions. 2. Perform pre-viva smoke testing on Reference Hardware (GTX 1650 Ti / 16GB RAM):
  - Execute complete offline dry-run with computer Wi-Fi completely disabled.
  - Validate that seed replay populates the database and triggers all 5 agents within 15–25 seconds for ≤ 50 posts.
  - Verify Ollama responds to analyst queries with cited document IDs without internet connection.
  - Verify sequential model loading releases VRAM cleanly (`torch.cuda.empty_cache()`) without OOM. 3. Finalize academic deliverables:
  - Comprehensive Project Report with literature review, methodology, and empirical evaluation.
  - Code documentation, docstrings, and type annotations across all modules.
  - Post-viva PII scrub verification script (`scripts/post_viva_scrub.sql`).
- **Final Deliverable:** Successful live external viva examination defense in December 2026.

---

4. Step-by-Step Implementation Prompts (Cursor / AI Workflow)

When developing with AI assistants or Cursor, execute the build strictly in this sequential order:

- **Step 0:** Repository Scaffolding, Python Virtual Environment & Docker Compose (`postgres:16-alpine` + `pgvector`, `redis:7-alpine`).
- **Step 1:** Database Schema & Declarative SQLAlchemy Models (`raw_posts`, `raw_authors`, `sentiment_results_b`, `demographic_aggregates_c`, `trend_topics_d`, `network_nodes_e`, `network_edges_e`, `rag_documents`, `rag_query_log`).
- **Step 2:** Component A Pyrofork MTProto Ingestion Adapter, Rate-Limit Jitter & Offline Demo Seed Script (`backfill.py --offline`).
- **Step 3:** Component B 3-Model Pipeline (`twitter-xlm-roberta`, `go_emotions`, `distilbart-mnli-12-3`) with sequential GPU execution and memory caching.
- **Step 4:** Component D BERTopic Topic Clustering, Velocity & Acceleration Calculation (capped at 500 docs).
- **Step 5:** Component C Demographic Heuristics & k-Anonymity Privacy Guards ($k \ge 50$).

- **Step 6:** Component E NetworkX Interaction Graph, Louvain Modularity & 3-Signal CIB Anomaly Detection.
- **Step 7:** Orchestrator Coordination Engine (15-min Row Runner for B, 48h Window Runner for C/D/E, and Instant Demo Trigger for ≤ 50 posts).
- **Step 8:** pgvector Semantic Search & Grounded RAG Engine (Ollama Air-Gapped Primary + Groq Cloud Optional).
- **Step 9:** React 18 Bento Dashboard & 7 Operations Views.
- **Step 10:** End-to-End Offline Viva Smoke Testing on GTX 1650 Ti Reference Hardware.

---

5. Non-Negotiable Engineering Invariants

1. **Offline-First Primary Product:** The offline demo (seed replay + Ollama) is the core product; live Telegram is optional depth. The entire system must demo with networking disabled.
2. **Hardware & VRAM Budget Discipline:** On 4 GB VRAM GPUs (GTX 1650 Ti), models must be executed sequentially (`torch.cuda.empty_cache()` between stages). Never attempt co-resident loading of all transformers + Ollama.
3. **Stance Detection Runtime Default:** Locked to `valhalla/distilbart-mnli-12-3` (~260 MB). BART-large is reserved for report benchmarks only.
4. **Demo Timing Constraint:** "Run Analysis Now" within 15–25 seconds is valid ONLY for a capped batch of ≤ 50 seed posts on the reference GPU.
5. **Temporal Anchoring:** All chronological metrics, velocities, and synchronies must be calculated strictly from platform `posted_at` UTC timestamps. Never use `ingested_at` for temporal analytics.
6. **Pyrofork Concurrency:** Ingestion worker concurrency is locked to `--concurrency=1` to prevent SQLite `.session` database lock corruption.
7. **DPDP Compliance ($k \ge 50$):** Demographic outputs must never identify individual citizens. Clusters below 50 accounts are suppressed (`below_threshold: true`).
8. **Advisory CIB Screening:** Coordinated behavior flags are advisory screening signals (0.0 to 1.0), never definitive legal verdicts.
9. **No Hallucination in RAG:** Cosine distance cutoff < 0.65 with short-circuit ensures the LLM is never called when relevant context is absent.
10. **Off-the-Shelf Model Honesty:** All models are pre-trained Hugging Face pipelines used zero-shot. Do not claim custom training or fine-tuning in any viva defense.

Document version: 4.8 (Implementation Plan & Engineering Roadmap) | Aligned with Project Brief v4.8 & Architecture Specification v4.8 | October 2026